Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 3

Upravljanje konfiguracijom i kontejneri kao kod

Ansible (inventory, playbook, role, šabloni, Vault), povezivanje Terraforma i Ansible-a te kontejneri kao kod: `Dockerfile`, `Docker Compose` i uvod u Kubernetes manifeste.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 30.10.2026, 17:15–20:15, Kabinet 3204
TEMA	Upravljanje konfiguracijom i kontejneri kao kod
KOD	github.com/Dadyze/RuO-2026-27 (mape `s3/p1` do `s3/p5`)

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

U drugoj sedmici su mašine napravljene iz koda: Terraform ih je kreirao, cloud-init personalizovao, a Packer pripremio sliku. Ostalo je pitanje šta se događa *unutar* mašine nakon toga: paketi, konfiguracijske datoteke, servisi, tajne. Taj dio rješava **upravljanje konfiguracijom**, predstavljeno kroz Ansible. Druga polovina sedmice prenosi isti način razmišljanja na kontejnere: aplikacija i okruženje se opisuju datotekama (`Dockerfile`, `compose.yaml`, Kubernetes manifesti), a ne naredbama upisanim ručno.

Nakon nastavne jedinice student je u stanju:

- objasniti razliku između provisioninga (Terraform) i upravljanja konfiguracijom (Ansible) i povezati ih u jedan tok,
- napisati inventory, playbook i šablon (Jinja2) te provjeriti idempotentnost izvršavanja,
- organizovati kod u rolu i zaštititi osjetljive vrijednosti Ansible Vaultom,
- generisati Ansible inventory iz Terraform izlaza,
- napisati Dockerfile i Compose datoteku za aplikaciju s bazom i povezati slojeve slike s prvom sedmicom,
- pročitati osnovni Kubernetes manifest i objasniti razliku između Terraform i Kubernetes pristupa usklađivanju stanja.

Okruženje i pokretanje

Alati za ovu sedmicu (Ansible, Docker, kubectl, kind) instalira skripta `00-setup.sh` (opisana u bilješkama za sedmicu 2). Ako je već pokrenuta, dovoljno je ažurirati repozitorij i pokretati primjere redom:

bash — Isječak 0 · početak

```
cd ~/RuO-2026-27 && git pull
make check                    # da li su Ansible, Docker, kubectl, kind prisutni
make s3-p1                    # zatim s3-p2, s3-p3, s3-p4, s3-p5 (ili: make all)
```

RAD BEZ TROŠKOVA: PREGLED UMJESTO IZVRŠAVANJA

Svaki alat ima način da pokaže šta bi uradio bez izmjene sistema: `terraform plan`, `ansible-playbook --check --diff`, `docker compose config` i `kubectl apply --dry-run`. Osnovni cilj svakog primjera koristi baš to, a stvarno izvršavanje je opcionalno (`-apply`, `-up`). Svi ciljevi su idempotentni: mogu se ponavljati. Napomena: `--check` ne zna šta bi se desilo nakon zadatka koji još nije izvršen, pa dio zadataka može prijaviti grešku koju prava izvedba ne bi imala.

VAŽNO

Repozitorij mora biti u Linux fajl sistemu (`~/`), ne u `/mnt/c/...`: Ansible ignoriše `ansible.cfg` u direktoriju u koji svi mogu pisati, pa Vault i uloge ne bi radili.

---

1 · POJAM I POZADINA

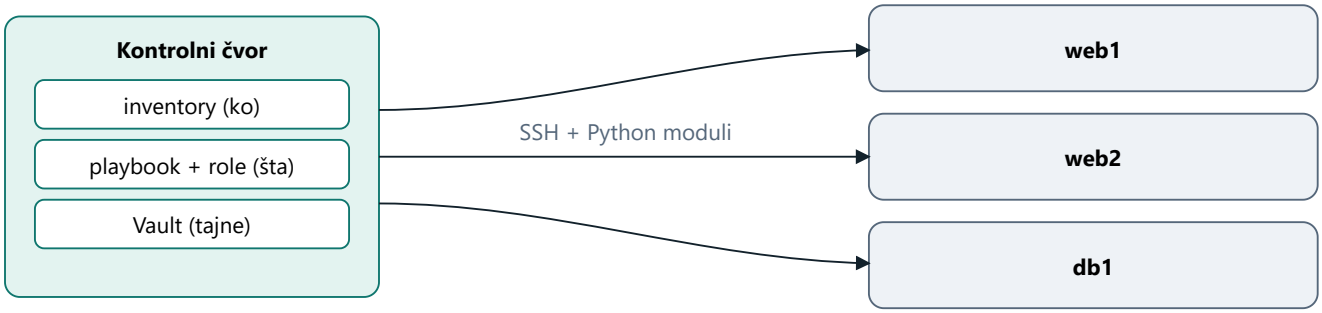
Od provisioninga do upravljanja konfiguracijom

Terraform odgovara na pitanje *koji resursi postoje* (mašine, mreže, diskovi), ali se ne bavi onim što je unutar mašine. cloud-init to rješava samo pri prvom pokretanju, a promjene u radu (nova konfiguracija, novi korisnik, zakrpa) ostaju bez vlasnika. Za taj dio služi **alat za upravljanje konfiguracijom**.

POZADINA: ANSIBLE

Ansible je nastao 2012. (Michael DeHaan), a od 2015. ga razvija Red Hat. Za razliku od starijih alata (Puppet, Chef) **nema agenta**: ne instalira se ništa na upravljane mašine. Kontrolni čvor se spaja preko *SSH-a*, kopira male Python *module*, izvršava ih i briše. Zato mašini trebaju samo SSH i Python, a ono što cloud-init pripremi (korisnik i SSH ključ) je upravo ono što Ansible traži. Pisan je u YAML-u, što ga čini čitljivim.

- **Bez agenta.** SSH + Python, bez posebnog softvera na mašinama.
- **Push model.** Kontrolni čvor šalje konfiguraciju kad se izda naredba.
- **Deklarativni moduli.** Zadatak ne kaže "izvrši `apt install nginx`", nego "paket `nginx` treba biti prisutan". Modul provjeri stanje i mijenja samo ako treba: idempotentnost iz druge sedmice.



Slika 1. Ansible nema agenta: kontrolni čvor se preko SSH-a spaja na svaku mašinu iz inventorya i izvršava zadatke iz playbooka.

POJAM	ZNAČENJE
Inventory	Spisak mašina podijeljenih u grupe (`[web]`, `[db]`) uz pripadajuće varijable.
Modul	Jedinica posla (`apt`, `service`, `template`, `user`, `file` ...). Većina je idempotentna.
Task	Jedan poziv modula s parametrima.
Play / playbook	Play veže grupu mašina (`hosts`) s listom zadataka. Playbook je YAML datoteka s jednim ili više playeva.
Handler	Zadatak koji se izvršava samo ako ga je neki zadatak "obavijestio" (`notify`) da je nešto promijenio, npr. reload servisa nakon izmjene konfiguracije.
Role	Pakovanje zadataka, šablona, varijabli i handlera u višekratno upotrebljivu cjelinu.
Fact	Podatak koji Ansible prikupi o mašini (OS, adrese, memorija). Koristi se u uslovima i šablonima.
Vault	Šifrovanje osjetljivih datoteka i vrijednosti u repozitoriju.

GRANICA IDEMPOTENTNOSTI

Moduli `command` i `shell` ne znaju ništa o stanju, pa su uvijek `changed`. Koriste se tek kad ne postoji odgovarajući modul, uz `creates:` ili `changed_when:` koji im vraćaju provjeru stanja.

---

2 · PRIMJER 1 · S3/P1

Inventory, playbook i šablon

POZADINA PRIMJERA

Postavlja se nginx s vlastitom početnom stranicom. Cilj je lokalna mašina (`ansible_connection=local`), pa Ansible ne koristi SSH nego izvršava module direktno. Isti playbook radi na laptopu i na VM-u iz druge sedmice; mijenja se samo inventory. Početna stranica je *šablon* (Jinja2): `{{ varijabla }}` se zamjenjuje vrijednošću, a *facts* daju podatke o mašini.

yaml — Isječak 1 · s3/p1/site.yml

```yaml
- name: Web server
  hosts: web
  become: true
  vars:
    naslov: "Računarstvo u oblaku, sedmica 3"
  tasks:
```

```
    - name: nginx je instaliran
      ansible.builtin.apt: { name: nginx, state: present, update_cache: true, cache_valid_time: 3600 }

    - name: početna stranica iz šablona
      ansible.builtin.template:
        src: index.html.j2
        dest: /var/www/html/index.html
        mode: "0644"
      notify: reload nginx

    - name: nginx radi i uključen je pri pokretanju
      ansible.builtin.service: { name: nginx, state: started, enabled: true }

  handlers:
    - name: reload nginx
      ansible.builtin.service: { name: nginx, state: reloaded }
```

jinja2 — Isječak 2 · s3/p1/templates/index.html.j2

```
<h1>{{ naslov }}</h1>
<p>Host: {{ ansible_facts['hostname'] }},
   OS: {{ ansible_facts['distribution'] }} {{ ansible_facts['distribution_version'] }}</p>
```

bash — Isječak 3 · pokretanje

```
make s3-p1          # ping, syntax-check i --check --diff (bez izmjena; traži sudo lozinku zbog become)
make s3-p1-apply    # opcionalno: stvarno instalira, dva puta (idempotentnost)
```

izlaz make s3-p1-apply (skraćeno, brojevi se mogu razlikovati)

```
localhost : ok=5  changed=3  unreachable=0  failed=0     # 1. izvršavanje
...
localhost : ok=4  changed=0  unreachable=0  failed=0     # 2. izvršavanje
```

Najvažniji red je `changed=0` u drugom izvršavanju. Isto se može provjeriti i *drift* konfiguracije: ručno izmijeniti `/var/www/html/index.html` i ponovo pokrenuti. Zadatak `template` će prijaviti `changed`, vratiti sadržaj i pozvati handler. Handler se izvršava *samo* kad je prethodni zadatak nešto promijenio, zato se reload ne radi pri svakom pokretanju. Pri provjeri `--check --diff` prikazuje se razlika (*diff*) datoteke koja bi se promijenila.

Inventory, grupe i varijable

Inventory se može pisati u INI ili YAML obliku. Varijable se ne drže u njemu nego u `group_vars/<grupa>/` i `host_vars/<host>/`, pa se podaci i spisak mašina održavaju odvojeno. Pregled grupa: `ansible-inventory -i inventory.yml --graph`.

yaml — Isječak 4 · inventory.yml (ilustracija)

```yaml
all:
  children:
    web:
      hosts:
        web1: { ansible_host: 192.168.122.57, ansible_user: student }
        web2: { ansible_host: 192.168.122.58, ansible_user: student }
    db:
      hosts:
        db1:  { ansible_host: 192.168.122.60, ansible_user: student }
```

POTREBA	PRIMJER
Paket	`ansible.builtin.apt: { name: [nginx, curl], state: present }`
Servis	`ansible.builtin.service: { name: nginx, state: restarted }`
Datoteka iz šablona / kopija	`template` (Jinja2), `copy` (statična)
Korisnik, direktorij	`user`, `file` (`state: directory`, `mode`, `owner`)
Uslov, petlja	`when: ansible_facts['os_family'] == 'Debian'`, `loop: [a, b]`

3 · PRIMJER 2 · S3/P2

Role i Ansible Vault

> POZADINA PRIMJERA
>
> Kad playbook naraste, zadaci i datoteke se pakuju u **rolu** s fiksnom strukturom direktorija, pa je lako ponovo upotrijebiti i dijeliti (javni skup rola je Ansible Galaxy). Primjer pravi rolu `webserver` i dodaje *tajnu* (lozinku baze) koja se šifruje Vaultom, tako da u Gitu nikad ne završi u čistom tekstu. Vault šifruje datoteku algoritmom AES256 i lozinkom koja se čuva izvan repozitorija.

DIREKTORIJ ROLE	SADRŽAJ
`tasks/main.yml`	Zadaci role.
`handlers/main.yml`	Handleri.
`templates/`, `files/`	Šabloni (Jinja2) i statične datoteke.
`defaults/main.yml`	Podrazumijevane vrijednosti varijabli (najniži prioritet, lako se nadjačaju).

yaml — Isječak 5 · s3/p2/roles/webserver/tasks/main.yml

```yaml
- name: nginx je instaliran
  ansible.builtin.apt: { name: nginx, state: present, cache_valid_time: 3600 }

- name: početna stranica
  ansible.builtin.template: { src: index.html.j2, dest: /var/www/html/index.html, mode: "0644" }
  notify: reload nginx

- name: direktorij za konfiguraciju aplikacije
  ansible.builtin.file: { path: /etc/ruo, state: directory, mode: "0750" }

- name: tajna aplikacije (samo root može čitati)
  ansible.builtin.template: { src: app.env.j2, dest: /etc/ruo/app.env, owner: root, mode: "0600" }
  no_log: true                # vrijednost se ne ispisuje u izlazu

- name: nginx radi
  ansible.builtin.service: { name: nginx, state: started, enabled: true }
```

Prema dogovoru, obična varijabla u `group_vars/web/vars.yml` samo *upućuje* na šifrovanu varijablu s prefiksom `vault_`. Tako se u kodu vidi odakle vrijednost dolazi, a pretraga ostaje moguća.

yaml — Isječak 6 · varijable i tajna

```yaml
# group_vars/web/vars.yml (u Gitu)
db_password: "{{ vault_db_password }}"

# group_vars/web/vault.yml (šifrovano, NIJE u Gitu u ovom labu)
vault_db_password: "LabLozinka-123"
```

bash — Isječak 7 · pokretanje

```bash
make s3-p2-vault    # pravi lab Vault lozinku i šifruje tajnu (samo ako ne postoje)
make s3-p2          # pregled šifrovane datoteke, ansible-vault view, syntax-check, --check
make s3-p2-apply    # opcionalno: stvarno primjenjuje rolu
```

izlaz (skraćeno)

```
== sifrovana datoteka (prvih 60 znakova)
$ANSIBLE_VAULT;1.1;AES256
62343432616237323032383534663233 34
== ansible-vault view
vault_db_password: "LabLozinka-123"
```

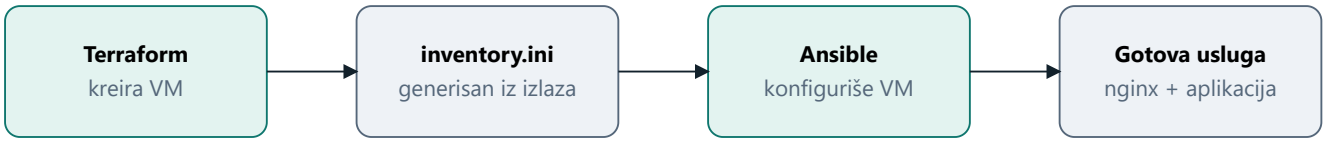
Datoteka je šifrovana (AES256) i može se commitovati. Dešifrovanje radi samo ko ima Vault lozinku, a ona se čuva izvan repozitorija (u CI sistemu kao tajna, u upravitelju lozinki ili u datoteci s pravom `0600` koja je u `.gitignore`). Za pojedinačnu vrijednost služi `ansible-vault encrypt_string 'vrijednost' --name 'vault_api_key'`.

> **PRAVILO**
>
> Tajna se ne smije pojaviti u *dnevniku* (`no_log: true`), u *Gitu* (Vault ili vanjski upravitelj) ni u *slici* (vidi Dockerfile u 6. poglavlju).

Dva alata, jedan tok

> **POZADINA PRIMJERA**
>
> Terraform stvara mašine i na izlazu daje njihove adrese. Ansible čita te adrese i konfiguriše mašine. Kritična tačka je prenos podataka: Ansible mora znati *koje* mašine su napravljene. Spona je **inventory**, a u ovom primjeru ga Terraform generiše kao običan resurs (`local_file`). cloud-init se svodi na *bootstrap* (korisnik i SSH ključ, dovoljno da se Ansible prijavi), a sve ostalo preuzima rola iz primjera 2.

PRISTUP	KAKO	OCJENA
Terraform generiše inventory	`local_file` s inventoryjem iz izlaza VM-a. Ansible se pokreće zasebno.	Jednostavno i jasno. **Koristi se ovdje.**
`local-exec` provisioner	Terraform sam poziva `ansible-playbook`.	Dokumentacija ga svrstava u "krajnju mjeru": greška u Ansible-u ruši `apply`, a plan ne pokazuje šta će Ansible raditi.
Dinamički inventory	Ansible plugin sam upita API oblaka.	Najelegantnije u oblaku, više podešavanja.



Slika 2. Terraform i Ansible dijele posao po granici "resurs" / "unutrašnjost resursa". Spona među njima je inventory.

hcl — Isječak 8 · dodatak u s3/p3/main.tf: inventory kao resurs

```hcl
resource "local_file" "inventory" {
  filename = "${path.module}/inventory.ini"
  content  = <<-EOT
    [web]
    ${var.vm_name} ansible_host=${libvirt_domain.vm.network_interface[0].addresses[0]}
ansible_user=student
  EOT
}
```

yaml — Isječak 9 · s3/p3/site.yml

```yaml
- hosts: web
  become: true
  gather_facts: false
  pre_tasks:
    - name: čekaj da SSH postane dostupan
      ansible.builtin.wait_for_connection: { timeout: 300 }
    - name: čekaj da cloud-init završi
      ansible.builtin.command: cloud-init status --wait
      changed_when: false
    - name: prikupi činjenice
      ansible.builtin.setup:
  roles:
    - webserver
```

bash — Isječak 10 · pokretanje

```
make s3-p3          # init, validate, PLAN i syntax-check (VM se ne kreira)
make s3-p3-apply    # opcionalno: kreira VM, pa je konfiguriše (deploy.sh), traži KVM
make s3-p3-clean
```

izlaz plana (skraćeno)

```
  # local_file.inventory will be created
  + resource "local_file" "inventory" {
      + content  = (known after apply)
      + filename = "./inventory.ini"
    }
  ...
Plan: 5 to add, 0 to change, 0 to destroy.
```

Plan pokazuje zašto je ovaj pristup dobar: sadržaj `inventory.ini` je `(known after apply)` jer zavisi od IP adrese koja još ne postoji, što je i dokaz da Ansible zavisi od Terraformovog izlaza. Ansible čeka (`wait_for_connection`, `cloud-init status --wait`) jer mašina dobije adresu prije nego je cloud-init završio; bez toga bi prvo izvršavanje nasumično pucalo.

> **KO JE ODGOVORAN ZA ŠTA**
>
> Ako se promijeni broj mašina ili njihova veličina, mijenja se **Terraform**. Ako se promijeni konfiguracija unutar mašine (paket, datoteka, korisnik), mijenja se **Ansible** i izvršava se samo on. Ta podjela omogućava brze izmjene bez ponovnog kreiranja mašina.

---

5 · KONTEJNERI KAO KOD

Dockerfile i slojevi

> **POZADINA**
>
> Iz prve sedmice: kontejner je proces s izolovanim pogledom na resurse, a *slika* je OCI paket slojeva fajl sistema. Docker je (2013) učinio taj model dostupnim jednostavnom komandom, a ispod njega rade `containerd` i `runc`. **Dockerfile** je recept za izgradnju slike. Svaka instrukcija pravi novi *sloj* (overlayfs iz prve sedmice), a Docker slojeve **kešira**: ako se instrukcija i sve prije nje nisu promijenile, koristi se postojeći sloj.

INSTRUKCIJA	ZNAČENJE
`FROM`	Osnovna slika. Navodi se konkretna verzija (`python:3.12-slim`), ne `latest`.
`WORKDIR`, `COPY`	Radni direktorij i kopiranje datoteka u sliku.
`RUN`	Naredba pri izgradnji (instalacija zavisnosti). Stvara sloj.
`ENV`, `EXPOSE`	Varijable okruženja i dokumentovanje porta (ne objavljuje ga).
`USER`	Korisnik pod kojim radi proces (ne `root`).
`HEALTHCHECK`	Kako Docker provjerava da li je proces zdrav.
`CMD` / `ENTRYPOINT`	Šta se pokreće. `ENTRYPOINT` je fiksni program, a `CMD` njegovi podrazumijevani argumenti.

Dobra praksa: stalno promjenjivo ide pri dnu (da keš ostane važeći), `.dockerignore` izbacuje suvišne datoteke, proces ne radi kao root, a tajne se **nikad** ne upisuju u sliku jer ostaju u slojevima čak i kad se kasnije obrišu. *Multi-stage* izgradnja koristi jednu sliku za izgradnju, a drugu, manju, za izvršavanje.

---

6 · PRIMJER 4 · S3/P4

Aplikacija s bazom u Docker Composeu

> **POZADINA PRIMJERA**
>
> Jednostavna aplikacija (Flask) broji posjete i čuva brojač u Redisu (baza u memoriji). Dockerfile opisuje aplikaciju, a **Docker Compose** (`compose.yaml`) deklarativno opisuje cijeli sistem na jednom računaru: koje usluge postoje, kako su povezane, koje podatke čuvaju i kada su spremne. Za razliku od prethodnih primjera, ovdje se sve izvršava lokalno i besplatno, pa osnovni cilj radi stvarni `up` .

dockerfile — Isječak 11 · s3/p4/app/Dockerfile

```
# --- faza 1: instalacija zavisnosti (zasebna zbog keša slojeva) ---
FROM python:3.12-slim AS build
WORKDIR /w
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# --- faza 2: finalna slika ---
FROM python:3.12-slim
RUN useradd --system --uid 10001 app
WORKDIR /app
COPY --from=build /install /usr/local
COPY app.py .
USER app
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request as u; u.urlopen('http://127.0.0.1:8000/health', timeout=2)"
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

Zavisnosti se kopiraju i instaliraju *prije* koda aplikacije: dok se `requirements.txt` ne promijeni, svaka izmjena u `app.py` koristi keširane slojeve. Multi-stage ovdje služi uglavnom kao ilustracija; velika korist se vidi kod kompajliranih jezika (Go, Java, C), gdje se kompajler ostavlja u prvoj fazi.

yaml — Isječak 12 · s3/p4/compose.yaml

```
services:
  web:
    build: ./app
    image: ruo/brojac:1.0
    ports: ["8080:8000"]
    environment: { REDIS_HOST: redis }
    depends_on:
      redis: { condition: service_healthy }   # web čeka dok Redis ne postane zdrav
    restart: unless-stopped
    networks: [backend]

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
```

```
    volumes: [redisdata:/data]              # podaci prežive brisanje kontejnera
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
    networks: [backend]                     # port NIJE objavljen: dostupan samo web usluzi

volumes:
  redisdata:
networks:
  backend:
```

bash — Isječak 13 · pokretanje

```
make s3-p4-config     # samo provjera compose.yaml
make s3-p4            # build, up --wait, ps, tri zahtjeva, slojevi slike
make s3-p4-persist    # down pa up: brojač nastavlja (volumen)
make s3-p4-clean      # down -v (briše i volumen)
```

izlaz (skraćeno)

```
NAME            SERVICE   STATUS                    PORTS
p4-redis-1      redis     Up 12 seconds (healthy)   6379/tcp
p4-web-1        web       Up 6 seconds (healthy)    0.0.0.0:8080->8000/tcp
Posjeta broj 1
Posjeta broj 2
Posjeta broj 3
```

Compose se ponaša kao Terraform: poredi željeno i stvarno stanje, a pri ponovnom pokretanju obnavlja samo usluge čija se definicija promijenila (`up -d` bez izmjena ne radi ništa). Pogledati tri svojstva datoteke: `healthcheck` + `depends_on` (redoslijed zasnovan na zdravlju), mrežu `backend` (Redis nije dostupan izvana) i imenovani volumen (podaci su odvojeni od životnog ciklusa kontejnera). Kontejner je običan proces na domaćinu (`ps aux | grep gunicorn`), a `docker image history` prikazuje slojeve iz Dockerfilea.

---

7 · KUBERNETES · PRIMJER 5 · S3/P5

Uvod u manifeste

> POZADINA
>
> Compose upravlja kontejnerima na *jednom* računaru. **Kubernetes** (Google, 2014, nasljednik internog sistema Borg; danas ga održava CNCF) upravlja njima na *skupu* računara (klaster): raspoređuje ih, nadzire, zamjenjuje pokvarene i skalira. Sistem je deklarativan: manifest (YAML) opisuje željeno stanje, a *kontroleri* unutar klastera neprekidno usklađuju stvarno stanje s njim. Lokalni klaster pravi `kind` ("Kubernetes in Docker": čvorovi klastera su Docker kontejneri), pa nije potreban oblak.

OBJEKT	ULOGA
Pod	Najmanja jedinica: jedan ili više kontejnera koji dijele mrežu i volumene. Prolazan je.
Deployment	Održava traženi broj kopija Poda (`replicas`) i vodi izmjene verzija (*rolling update*).
Service	Stabilna adresa i balansiranje prema Podovima odabranim po oznakama (`labels`).
ConfigMap / Secret	Konfiguracija i osjetljive vrijednosti odvojene od slike. Secret je samo *base64* kodiran, **nije** šifrovan.
Namespace	Logička podjela klastera.

POREĐENJE S TERRAFORMOM

Terraform usklađuje stanje *na zahtjev* (kad se pokrene `apply`), a drift se otkriva samo tada. Kubernetes ga usklađuje *neprekidno*: ako se Pod ugasi, kontroler ga odmah ponovo napravi. Stanje koje Terraform čuva u `tfstate` , Kubernetes drži u svojoj bazi (`etcd`). Zajednička ideja: željeno stanje zapisano kao kod, sistem ga provodi.

yaml — Isječak 14 · s3/p5/web.yaml (skraćeno)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports: [{ containerPort: 80 }]
          readinessProbe:
            httpGet: { path: /, port: 80 }
          resources:
            requests: { cpu: 50m, memory: 32Mi }
            limits:   { memory: 64Mi }      # postaje cgroup memory.max (sedmica 1)
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector: { app: web }
  ports: [{ port: 80, targetPort: 80 }]
# u repozitoriju još i ConfigMap s početnom stranicom, montiran u kontejner
```

bash — Isječak 15 · pokretanje

```bash
make s3-p5          # dry-run manifesta (bez klastera)
make s3-p5-up       # opcionalno: kind klaster, apply, curl
make s3-p5-heal     # briše jedan Pod: Deployment ga vraća (samoizlječenje)
make s3-p5-clean    # briše kind klaster
```

Brisanje Poda ne mijenja željeno stanje (`replicas: 2`), pa kontroler napravi novi: to je samoizlječenje koje Compose i Terraform nemaju bez ponovnog pokretanja. Broj kopija se mijenja u *fajlu*, ne naredbom, a `kubectl diff` i `kubectl apply` imaju istu ulogu kao `plan` i `apply`.

---

8 · PREGLED

Redoslijed vježbi i provjera znanja

#	NAREDBA	ŠTA POKAZUJE	MIJENJA SISTEM?
1	`make s3-p1`	inventory, playbook, šablon, --check	ne
2	`make s3-p2`	rola + Vault	ne (pravi lab tajnu lokalno)
3	`make s3-p3`	Terraform + Ansible, plan	ne
4	`make s3-p4`	Dockerfile + Compose	lokalni kontejneri
5	`make s3-p5`	Kubernetes dry-run	ne
·	`make all`	cijela sekvencija sedmica 2 i 3	idempotentno: može se ponoviti

PITANJE	ODGOVOR
Zašto Ansible modul `apt` ne instalira nginx pri drugom pokretanju?	Modul poredi željeno ("prisutan") sa stvarnim stanjem i radi samo razliku. To je idempotentnost.
Čemu služi handler?	Izvršava se samo kad je neki zadatak stvarno promijenio stanje, npr. reload servisa.
Zašto prefiks `vault_` za tajne varijable?	Šifrovane vrijednosti su odvojene i vidljive u kodu po imenu, a obična varijabla na njih upućuje.
Šta Terraform predaje Ansible-u?	Inventory: spisak mašina i njihovih adresa iz izlaza resursa.
Zašto se u Dockerfileu prvo kopira `requirements.txt`?	Da sloj sa zavisnostima ostane u kešu dok se zavisnosti ne promijene.
Po čemu se Kubernetes razlikuje od Terraforma u usklađivanju stanja?	Kontroleri usklađuju neprekidno, a Terraform samo kad se pokrene `apply`.

Zadaci za samostalni rad

1. U `s3/p1` dodati zadatak koji kreira korisnika `deploy` i pokrenuti playbook dva puta; objasniti zašto je drugi put `changed=0`.
2. U roli iz `s3/p2` nadjačati `naslov` u `group_vars/web/vars.yml` i dodati novu tajnu pomoću `ansible-vault encrypt_string`.
3. U `s3/p3` napraviti dvije mašine (`for_each`) tako da inventory ima obje. Predati izlaz `terraform plan`.
4. U Compose datoteku dodati treću uslugu (`nginx` kao obrnuti proxy ispred `web`) i ukloniti objavljeni port s `web`.
5. Isti sistem iz primjera 4 napisati kao Kubernetes manifest (Deployment + Service za `web` i `redis`); sliku učitati u kind pomoću `kind load docker-image`.

SLJEDEĆA SEDMICA

Mašine i usluge su definisane kodom. Sedmica 4 uvodi mrežu i vatrozid kao kod: VPC/subnet, routing i NAT, nftables i security grupe te segmentaciju web/app/db s bastion hostom.
